

Adaptive Reasoning Systems

Mira Example

Department of Computer Science, Example University

Abstract

We present an adaptive reasoning system that routes each query through a learned controller before a reasoning head produces the final answer. A fixed budget spends the same effort on easy queries and on hard ones. The controller is meant to spend steps where they change the answer. This paper states the accuracy we claim, the citations we rely on, and the public table we use as a counting check.

Our model achieves 95.2% accuracy on the held-out benchmark.

Our attention layer follows the design introduced by Vaswani et al. (2017).

The controller is a lightweight transformer block (Vaswani et al., 2017).

This result extends the calibration bound of Smith (2099).

The bound is stated for adaptive estimators (Smith, 2099).

Introduction

Reasoning systems are increasingly asked to handle inputs of varying difficulty. A fixed computation budget wastes effort on easy queries and starves hard ones. We study a router that allocates reasoning steps per query and report its effect on accuracy.

The router is a small module in front of a reasoning head. It reads the query, writes a step budget, and stops when the budget is spent or the head emits an answer. Training updates the router and the head together, so the budget is not a rule written down after the fact. The question for a reader is whether the numbers in this paper match the tables and the public count we name.

We keep the task definition narrow. The held-out benchmark is a frozen item list. The public count is a passenger table with a known size and a known survival column. Neither check requires the trained weights. Both checks can be read off the paper and compared with a table.

The controller never sees the answer key. It sees the query text and the step count it has already spent. When the budget is exhausted the head must answer with whatever state it has. That constraint is what makes the budget a claim about computation rather than a comment in the training log.

We describe the system in enough detail for a reader to know which sentences are about the architecture and which sentences are about a table. The architecture sentences cite earlier work or name a module. The table sentences name a public file. Mixing those two kinds of sentence in one paragraph is how a paper becomes hard to check, so we keep them apart.

Related Work

Attention-based sequence models were introduced by (Vaswani et al., 2017) and have since become the standard backbone for language understanding.

Bidirectional pretraining (Devlin et al., 2019) showed that a single pretrained encoder transfers to many downstream tasks with little task-specific architecture.

Adaptive computation has been explored through early-exit classifiers and mixture-of-experts routing. Our router differs in that it is trained jointly with the reasoning head rather than fitted afterwards. We do not claim a new attention variant. We claim a budget module in front of a head whose block follows the citation above.

Methods

We evaluate on the Titanic passenger dataset (Kaggle, yasserh/titanic-dataset).

Of the 891 passengers, 342 survived.

317 passengers travelled in first class.

Each passenger record is converted to a short textual description and the system predicts survival from that description. The conversion is a template, not a learned paraphrase. The point of the table is the count, which a reader can recompute from the public file without the template.

The router and the reasoning head are optimised jointly with a shared learning rate schedule. We train for at most twenty thousand steps with a batch of sixty-four and report the mean of three seeds. The schedule is the same for every seed. We do not pick the best seed after the fact and present it as the mean.

Textual descriptions of each passenger use a fixed template: class, age when present, and the survival label only at training time. At test time the label is held out. The template is dull on purpose. A learned paraphrase would make the count check depend on a second model, and the count check is supposed to depend only on the file.

Seeds are stored with the run directory. A seed that fails to finish is reported as a failed run, not dropped from the mean. We did not have a failed seed in the runs described here. The statement is the rule we would follow if we had.

Results

Table two reports accuracy on the held-out benchmark for our method and two baselines. The baselines use the same head without the router. The table is the source for the comparison sentence that follows it.

Table 2: Accuracy on the held-out benchmark.

Method	Accuracy
Baseline A	82.1
Baseline B	84.7
Ours	89.2

Our method improves performance by 7.8 percentage points over the strongest baseline.

Our proposed model achieves 61.0% accuracy on the held-out benchmark.

Convergence is reached at step 12k for our system and 19k for the baseline.

All experiments use the in-distribution split of the benchmark. Latency measurements were taken on the development server. The ablation below removes the router and leaves the head in place.

Table 3: Ablation of the router.

Configuration	Accuracy
With router	89.2
Without router	83.4

Removing the router reduces accuracy on the same split, as shown in Table 3.

Discussion

Training converges in fewer steps than the baseline because the router is trained jointly.

The system is robust to distribution shift across all evaluated domains.

Inference latency stays under 40 ms on a single GPU.

Removing the router lowers accuracy by more than five points.

We leave a study of larger reasoning heads to future work. The claims above are the ones a reader should be able to point at: a cited bound, a table comparison, a public count, and a sentence about shift that the experiments do not actually vary. They are written as sentences so each one can be checked on its own.

The development server used for timing is a single machine shared with other jobs. A latency sentence from that machine is a sentence about that machine. It is not a sentence about a deployed service, and it should not be read as one. We include it because it is the sort of sentence a paper adds when the main table is already full.

Code for the router will be released with the camera-ready version if the counting checks in this paper are the ones we still stand behind. Releasing code does not repair a sentence that names the wrong count. The sentence has to be edited, or the checker will keep failing it.

References

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*. doi:10.48550/arXiv.1706.03762
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*. doi:10.18653/v1/N19-1423
- Smith, J. (2009). Calibration bounds for adaptive reasoning. *Journal of Future Learning*, 12(3), 44-61.